

A Neural Algorithm of Artistic Style

A Summary of Gatys, Ecker & Bethge (2015), with Mathematical Historical Context

The idea

A painting has two things. It has content and it has style. The content is what is shown. The style is how it is shown. Gatys, Ecker, and Bethge pulled the two apart with a convolutional network trained only to recognize objects. They kept the content of a photograph, kept the style of a painting, and joined them in one image [8]. They trained nothing new. The network was fixed. The image was the variable.

The method looks sudden, but it is not. It is the meeting of two older lines of mathematics. One asks how to describe a texture by its statistics. The other asks how to rebuild an image from the features a network extracts. Gatys stood exactly where the two lines cross. To see the paper clearly, walk each line first.

The first line: texture as statistics

The oldest idea is that a texture is not a fixed arrangement of pixels but a set of statistics. Julesz asked when two textures look the same without close inspection [1]. His conjecture was that they match when their low-order joint pixel statistics match. Write $\phi(I)$ for a chosen vector of statistics of an image I . Then a texture is not one image but an equivalence class,

$$\mathcal{T}(I_0) = \{ I : \phi(I) = \phi(I_0) \}.$$

Later counterexamples showed second-order pixel statistics are not enough, and the search moved to richer ϕ [1]. The frame held. Only ϕ changed.

Heeger and Bergen chose ϕ from a filter bank [2]. They decomposed an image into a steerable pyramid, a set of band-pass responses $\psi_s * I$ across scales and orientations s , and matched the marginal histograms of those responses,

$$h(\psi_s * I) = h(\psi_s * I_0) \quad \text{for all } s.$$

They synthesized by alternating histogram matching in the pyramid domain and the pixel domain. These are first-order statistics of filter outputs. They capture roughness and grain but miss how structures line up.

Portilla and Simoncelli fixed that [3]. They built ϕ from the *joint* statistics of complex wavelet coefficients $\{a_s\}$: marginal moments, and, crucially, the correlations between coefficients across position, orientation, and scale,

$$C_{s,s'} = \mathbb{E}[a_s a_{s'}^*].$$

Several hundred such numbers described a texture well enough to resynthesize it by iterative projection onto $\{ I : \phi(I) = \phi(I_0) \}$. The heart of the model was the pairwise product of filter responses. Remember that term. It returns.

The last step on this line was taken by the authors themselves, one paper before the style paper [7]. They kept Portilla and Simoncelli's idea and changed the filters. Instead of hand-designed wavelets, they used the feature maps of a deep network. Collect the responses at layer l in a matrix $F^l \in \mathbb{R}^{N_l \times M_l}$, with N_l channels and M_l spatial positions. The texture statistic is the Gram matrix, the correlation between channels,

$$G_{ij}^l = \sum_k F_{ik}^l F_{jk}^l = \langle f_i^l, f_j^l \rangle.$$

This is the pairwise product again, now over learned filters rather than wavelets. To synthesize a texture they minimized the mismatch of Gram matrices across layers,

$$x^* = \arg \min_x \sum_l w_l \|G^l(x) - G^l(I_0)\|_F^2.$$

The style loss of the famous paper is already here, in full.

The second line: content as invertible features

The other line asks the reverse question. Given the features a network computes, can you recover an image? Mahendran and Vedaldi showed you can [6]. For a representation Φ and a target $\Phi_0 = \Phi(I_0)$, they reconstructed an image by

$$x^* = \arg \min_x \|\Phi(x) - \Phi_0\|_2^2 + \lambda \mathcal{R}(x),$$

with $\mathcal{R}$ a natural-image regularizer such as total variation. Two facts came out of it. The pixels can be chased by gradient descent to match a target in feature space. And the deeper the layer, the more the reconstruction keeps the meaning of the scene while losing its exact pixels. Depth trades geometry for abstraction.

This rested on a known substrate. Visualization studies had shown that a trained network builds a hierarchy: edges low, textures above them, parts and objects near the top [4]. VGG-19 gave Gatys a deep, clean version of that stack [5]. So "content lives in the deep layers" was not a guess. It was read off the hierarchy.

The confluence: the Gatys objective

Now the two lines meet. Content is a feature inversion at a deep layer, in the manner of Mahendran and Vedaldi. Style is a Gram-matrix match across layers, in the manner of the texture model. Gatys put both into one objective and optimized the pixels against it [8].

Content first. Let $\vec{p}$ be the content photograph and $\vec{x}$ the generated image, with feature matrices P^l and F^l at layer l . The content loss is the feature-inversion term,

$$\mathcal{L}_{\text{content}}(\vec{p}, \vec{x}, l) = \frac{1}{2} \sum_{i,j} (F_{ij}^l - P_{ij}^l)^2,$$

with the simple gradient, zero on the dead units,

$$\frac{\partial \mathcal{L}_{\text{content}}}{\partial F_{ij}^l} = \begin{cases} (F^l - P^l)_{ij} & F_{ij}^l > 0, \\ 0 & F_{ij}^l < 0. \end{cases}$$

Match a high layer and you match the layout, not the exact pixels. That is the point of choosing depth.

Style next. Let $\vec{a}$ be the artwork, with Gram matrices A^l , and G^l the Gram matrices of $\vec{x}$. The per-layer error and the total style loss are the texture terms,

$$E_l = \frac{1}{4N_l^2 M_l^2} \sum_{i,j} (G_{ij}^l - A_{ij}^l)^2, \quad \mathcal{L}_{\text{style}}(\vec{a}, \vec{x}) = \sum_l w_l E_l.$$

The Gram matrix throws away position and keeps correlation. It keeps texture, color, and stroke, and does not care where the stroke sits. That is why it reads as style and not content.

Add the two, weighted,

$$\mathcal{L}_{\text{total}}(\vec{p}, \vec{a}, \vec{x}) = \alpha \mathcal{L}_{\text{content}}(\vec{p}, \vec{x}) + \beta \mathcal{L}_{\text{style}}(\vec{a}, \vec{x}).$$

The ratio α/β sets the balance. Push α up and the photograph shows through. Push β up and the paint takes over.

The network and the search

The network is VGG-19: sixteen convolutional layers and five pooling layers, the fully connected layers dropped [5]. Content is read at `conv4_2`. Style is read at `conv1_1` through `conv5_1`, low and high at once, to catch both fine grain and broad pattern. Gatys swapped max pooling for average pooling; the gradients flowed better and the images looked cleaner.

The search inherits the machinery of both lines. There is no trained generator. The image is the parameter. Start from white noise, run it through the fixed network, compute the loss, backpropagate to the pixels, and step with L-BFGS. The same gradients that train networks are used here, but the weights are frozen and the picture moves.

What it showed, and what it cost

Two results matter. Content and style are separable inside a recognition network; the split was not designed but fell out of training on objects. And once separated, the two recombine, so any photo can wear the style of any painting.

There is a clean way to see why the Gram matrix works as a style measure. Later analysis showed that matching Gram matrices is equivalent to minimizing the maximum mean discrepancy between the two feature distributions under a second-order polynomial kernel [9]. Style matching is distribution matching. The Gram matrix was the right statistic for a reason.

The cost is speed. Each image is a fresh optimization, many iterations, minutes on a GPU, nothing reused. The content–style trade-off is real; both cannot be matched to the letter, and α/β names your side of it. Resolution is capped by memory. This cost is the question handed to the next decade. Gatys made the transfer beautiful and left it expensive. The feed-forward and adaptive- normalization methods that followed are answers to that single word: expensive.

References

- [1] B. Julesz. Visual Pattern Discrimination. IRE Transactions on Information Theory, 1962. (See also Julesz, Textons, the Elements of Texture Perception, Nature, 1981.)
- [2] D. J. Heeger and J. R. Bergen. Pyramid-Based Texture Analysis/Synthesis. SIGGRAPH, 1995.
- [3] J. Portilla and E. P. Simoncelli. A Parametric Texture Model Based on Joint Statistics of Complex Wavelet Coefficients. IJCV, 2000.
- [4] M. D. Zeiler and R. Fergus. Visualizing and Understanding Convolutional Networks. ECCV, 2014.
- [5] K. Simonyan and A. Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition (VGG). ICLR, 2015.
- [6] A. Mahendran and A. Vedaldi. Understanding Deep Image Representations by Inverting Them. CVPR, 2015.
- [7] L. A. Gatys, A. S. Ecker, and M. Bethge. Texture Synthesis Using Convolutional Neural Networks. NIPS, 2015.
- [8] L. A. Gatys, A. S. Ecker, and M. Bethge. A Neural Algorithm of Artistic Style. arXiv:1508.06576, 2015. (Published as Image Style Transfer Using Convolutional Neural Networks, CVPR, 2016.)
- [9] Y. Li, N. Wang, J. Liu, and X. Hou. Demystifying Neural Style Transfer. IJCAI, 2017.